

Segment Trees in Competitive Programming

Range Queries, Point Updates, and Laziness

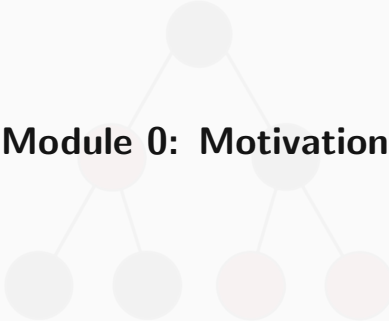


KCPC Training

Overview

- 1 Module 0: Motivation
- 2 Module 1: Segment Tree Intuition and Queries
- 3 Module 2: Point Updates
- 4 Module 3: Build
- 5 Module 4: Let's be Lazy With this Problem





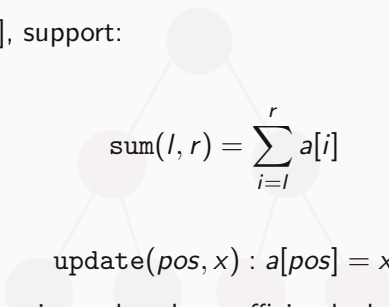
Module 0: Motivation

The Core Problem

We begin with a very common task in competitive programming with *queries*:

Given an array $a[0 \dots n - 1]$, support:

→ **Range sum query**


$$\text{sum}(l, r) = \sum_{i=l}^r a[i]$$

→ **Point update**

$$\text{update}(\text{pos}, x) : a[\text{pos}] = x$$

We want to answer many queries and updates efficiently, hopefully 10^5 of them. So, it's a simple trade-off question:

Can we make both operations fast at the same time?

Naïve Approach

The simplest idea:

- For $\text{sum}(l, r)$, loop from l to r
- For $\text{update}(pos, x)$, just write $a[pos] = x$

Complexity:

- Query: $\mathcal{O}(r - l + 1)$, worst-case $\mathcal{O}(n)$
- Update: $\mathcal{O}(1)$

If we have q operations and many of them are long queries, total work becomes:

$$\mathcal{O}(qn) \approx \mathcal{O}(n^2) \text{ if } q \approx n$$

Too slow for $n, q \approx 10^5$.

Prefix Sums

If the array were **static**, prefix sums would solve range sums perfectly. Define prefix sum as a cumulative array:

$$\text{pref}[i] = a[0] + a[1] + \cdots + a[i]$$

Then:

$$\text{sum}(l, r) = \begin{cases} \text{pref}[r] & l = 0 \\ \text{pref}[r] - \text{pref}[l - 1] & l > 0 \end{cases}$$

→ Query: $\mathcal{O}(1)$

→ Update: $\mathcal{O}(n)$, because all later prefix values change

So prefix sums are great for **read-only arrays**, but bad for **dynamic arrays**.

The Trade-Off

So far we have two extremes:

- **Plain array**
 - update fast
 - query slow
- **Prefix sums**
 - query fast
 - update slow

What we really want is:

- logarithmic everything
- (or constant! but we won't have that today, unfortunately)

The key is to store answers as always but not just for prefixes, also for many intervals.



Module 1: Segment Tree Intuition and Queries



Main Intuition

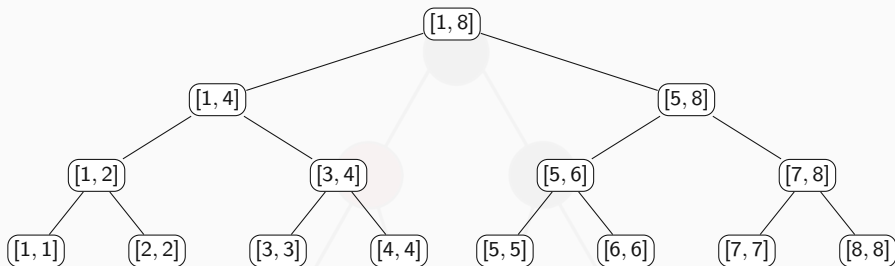
Suppose we somehow knew:

- the sum of the whole array
- the sum of the left half and right half
- the sum of each quarter
- the sum of each eighth
- and so on...

Then many range queries could be answered by combining only a few precomputed pieces. Like how any natural number can be written in binary... This is exactly the **segment** tree's idea:

- each node corresponds to an interval
- each node stores the answer for that interval
- the interval is split into left half and right half

Tree of Intervals



- The root represents the whole array.
- Leaves represent single elements.
- Internal nodes merge the answers of their children.

How a Query Uses the Tree

Suppose we want:

$$\text{sum}(2, 7)$$

We may not have one node exactly equal to $[2, 7]$, but we can decompose it into a few stored segments:

$$[2, 2] \cup [3, 4] \cup [5, 6] \cup [7, 7]$$

So:

$$\text{sum}(2, 7) = \text{val}([2, 2]) + \text{val}([3, 4]) + \text{val}([5, 6]) + \text{val}([7, 7])$$

The segment tree finds this **decomposition** quickly.

Three Query Cases

Suppose a node covers $[tl, tr]$, and let

$$tm = \left\lfloor \frac{tl + tr}{2} \right\rfloor$$

For a query interval $[l, r]$, there are three possibilities:

- **Exact match:** $[l, r] = [tl, tr]$
Return the precomputed answer immediately.
- **Entirely in one child:**
Recurse into left or right child only.
- **Intersects both children:**
Split into two subqueries and merge the answers.

For sums, the merge operation is simply addition.

Why is Query Time $\mathcal{O}(\log n)$?

The height of the tree is:

$$\mathcal{O}(\log n)$$

because the interval length is halved at every level.

Also, a range query only branches near its two boundaries:

- one branch approaches the left boundary
- one branch approaches the right boundary
- fully covered middle segments stop immediately

So each level contributes only a constant number of useful nodes.

Therefore:

$$\text{query} = \mathcal{O}(\log n)$$

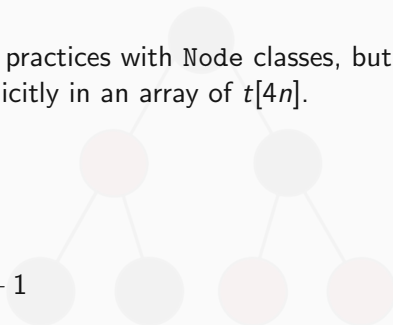
How We Store the Tree

You could have some OOP practices with Node classes, but we are better than that: store the segment tree implicitly in an array of $t[4n]$.

Convention:

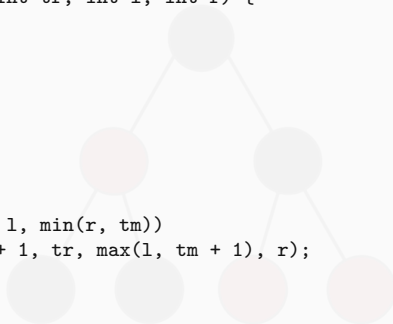
- root at index 1
- left child of v is $2v$
- right child of v is $2v + 1$

The interval $[tl, tr]$ of a node is passed as function parameters.



Range Sum Query: Pseudocode

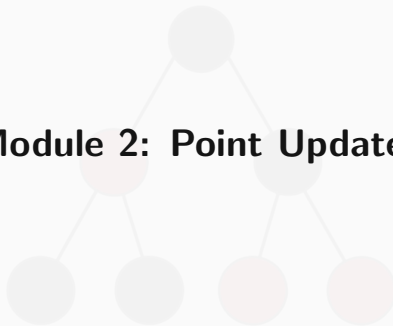
```
long long sum(int v, int tl, int tr, int l, int r) {  
    if (l > r)  
        return 0;  
  
    if (l == tl && r == tr)  
        return t[v];  
  
    int tm = (tl + tr) / 2;  
  
    return sum(v * 2, tl, tm, l, min(r, tm))  
        + sum(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r);  
}
```



Idea:

- if the current node exactly matches the query, use it
- otherwise ask only the necessary children

Module 2: Point Updates

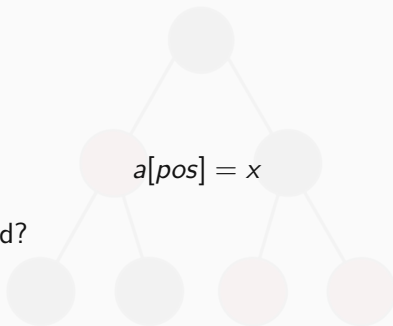


We Know How to Query — But How Do We Update?

Suppose we perform:

$$a[pos] = x$$

Which nodes become invalid?



Update Intuition

Only the nodes whose intervals contain pos .

That means:

- one leaf changes
- one node on each level lies on the path to that leaf
- all other nodes remain correct

So we only need to update a *single root-to-leaf path*.

Update strategy:

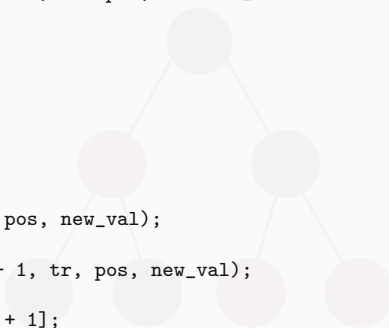
- 1 descend to the leaf representing $a[pos]$
- 2 overwrite its value
- 3 recompute every ancestor while returning upward

So:

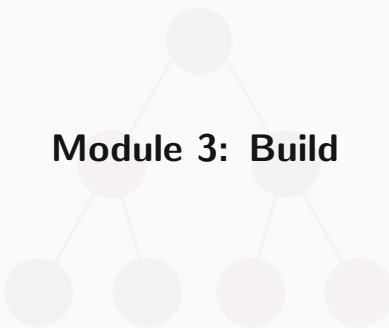
$$\text{update} = \mathcal{O}(\log n)$$

Point Update: Pseudocode

```
void update(int v, int tl, int tr, int pos, int new_val) {  
    if (tl == tr) {  
        t[v] = new_val;  
        return;  
    }  
  
    int tm = (tl + tr) / 2;  
  
    if (pos <= tm)  
        update(v * 2, tl, tm, pos, new_val);  
    else  
        update(v * 2 + 1, tm + 1, tr, pos, new_val);  
  
    t[v] = t[v * 2] + t[v * 2 + 1];  
}
```



- Leaf: directly store the new value.
- Internal node: recompute using the merge operation.



Module 3: Build

Finally: How Do We Build the Tree?

So far, we assumed the tree already exists.

To build it:

- each leaf gets its array value
- each internal node is merged from its two children

For a sum segment tree:

$$t[v] = t[2v] + t[2v + 1]$$

This is naturally done with recursion.

Build Intuition

Conceptually, build is bottom-up:

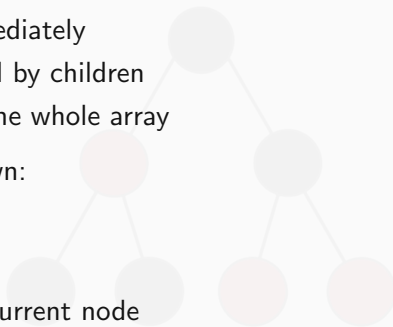
- leaves are known immediately
- parents are determined by children
- the root summarizes the whole array

In code we write it top-down:

- 1 build left child
- 2 build right child
- 3 merge them into the current node

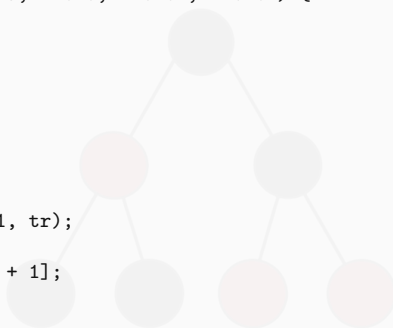
Each node is processed once, so:

$$\text{build} = \mathcal{O}(n)$$



Build: Pseudocode

```
void build(vector<long long>& a, int v, int tl, int tr) {  
    if (tl == tr) {  
        t[v] = a[tl];  
        return;  
    }  
  
    int tm = (tl + tr) / 2;  
  
    build(a, v * 2, tl, tm);  
    build(a, v * 2 + 1, tm + 1, tr);  
  
    t[v] = t[v * 2] + t[v * 2 + 1];  
}
```



Main call:

```
build(a, 1, 0, n - 1);
```

Basic Segment Tree Summary

Method	Query	Update
Naïve array	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Prefix sums	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Segment tree	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

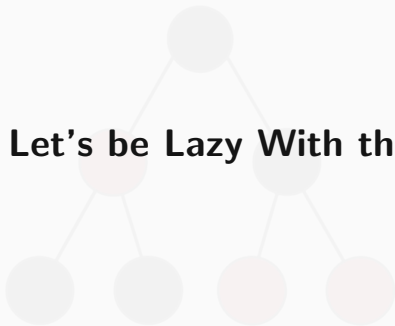
Memory:

$\mathcal{O}(n)$

usually implemented as an array of size $4n$. The same framework works for:

- sum
- minimum / maximum
- gcd
- counts
- and many, many more

Module 4: Let's be Lazy With this Problem



Problem E: A Simple Task

We are given a string S of length n , and q queries.

Each query is:

(l, r, k)

meaning:

- if $k = 1$, sort the substring $S[l \dots r]$ in **non-decreasing** order
- if $k = 0$, sort the substring $S[l \dots r]$ in **non-increasing** order

Then output the final string after all queries.

Constraints are large:

$$n \leq 10^5, \quad q \leq 5 \cdot 10^4$$

So before we code anything, ask:

What information do we really need in order to sort a substring?

First Thought: Just Sort the Substring

A direct idea:

- extract $S[l \dots r]$
- sort it
- write it back

Or, since the alphabet is only lowercase English letters, use **counting sort**:

- count how many a, b, ..., z are in the substring
- rewrite the substring in sorted order

For one query this costs: $\mathcal{O}(r - l + 1 + 26)$

In the worst case that becomes: $\mathcal{O}(nq)$, which is far too slow for 10^5 -scale input.

A Better Way to Think About Sorting

Suppose we want to sort $S[l \dots r]$ increasingly. We actually do **not** care about the old order inside the substring. We only care about the **frequency** of each character. So every query is really:

- 1 count how many times each character appears in $S[l \dots r]$
- 2 rebuild the substring as blocks:

aaa ... bbb ... zzz

or the reverse order if $k = 0$

Example: cbaba $\xrightarrow{\text{sort increasing}}$ aabbc

So the real problem is no longer “sorting”. It is:

count letters on a range, then rewrite whole blocks quickly.

What Operations Would Make This Easy?

For each character $c \in \{a, \dots, z\}$, define a binary array:

$$b_c[i] = \begin{cases} 1 & \text{if } S[i] = c \\ 0 & \text{otherwise} \end{cases}$$

Then for each character we would like to support:

- **Range sum query:** how many copies of c are in $[l, r]$?
- **Range assignment:** set an entire interval to all 0s or all 1s

If we had these two operations, then sorting a substring would become straightforward.

26 Segment Trees

This suggests the following structure:

- build **26 segment trees**
- tree c stores the binary array for character c
- node value = number of occurrences of that character in the segment

So for a query $[l, r]$:

- query all 26 trees to get the counts
- clear the whole interval $[l, r]$
- write back character blocks in increasing or decreasing order

This is a *beautiful* plan.

But there is one missing ingredient:

how do we update a whole range quickly?

Okay — If We Had This Range Update, We Would Be Done

In one character tree, we need:

- $\text{sum}(l, r)$: count 1s in a range
- $\text{assign}(l, r, 0)$: erase a block
- $\text{assign}(l, r, 1)$: fill a block

A normal segment tree already gives us fast range sums.

The problem is the assignment. If we assign position by position, then rewriting a long substring is expensive again.

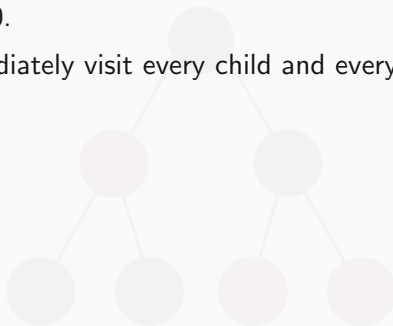
So the next question is:

*Can we delay work, but still keep the answers correct? Can we be **lazy**?*

Lazy Propagation Intuition

Suppose a node covers the interval $[tl, tr]$, and we learn that **every value in this whole interval becomes 0**.

Do we really need to immediately visit every child and every leaf?



Lazy Propagation Intuition

Suppose a node covers the interval $[tl, tr]$, and we learn that **every value in this whole interval becomes 0**.

Do we really need to immediately visit every child and every leaf?

No.

Instead, we can:

- immediately fix the answer for the current node
- store a small note saying:
"if anyone later asks about my children, remember that they should all become 0"
- only push that information downward when it becomes necessary

This is exactly the idea of **lazy propagation**:

postpone detailed updates until they are actually needed

What a Node Stores in One Character Tree

For one fixed character:

- $t[v]$ = how many 1s are in this segment
- $lazy[v]$ stores a pending assignment

For the lazy tag we use:

- -1 : no pending update
- 0 : this whole segment should be all zeros
- 1 : this whole segment should be all ones

If a segment $[tl, tr]$ is entirely assigned to $x \in \{0, 1\}$, then:

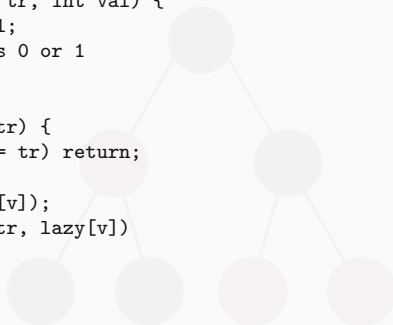
$$t[v] = (tr - tl + 1) \cdot x$$

That is enough to answer sums on this whole segment immediately, without touching the children yet.

Lazy Propagation: apply and push

```
void apply(int v, int tl, int tr, int val) {  
    t[v] = (tr - tl + 1) * val;  
    lazy[v] = val;    // val is 0 or 1  
}
```

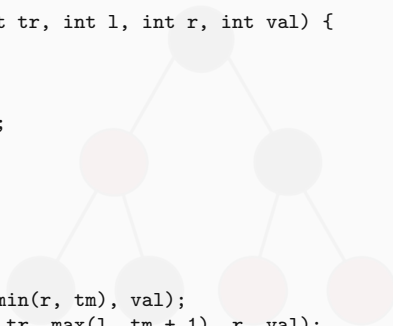
```
void push(int v, int tl, int tr) {  
    if (lazy[v] == -1 || tl == tr) return;  
    int tm = (tl + tr) / 2;  
    apply(v * 2, tl, tm, lazy[v]);  
    apply(v * 2 + 1, tm + 1, tr, lazy[v]);  
    lazy[v] = -1;  
}
```



- apply updates the current segment immediately
- push forwards postponed information to the children
- we only push when we need to go deeper

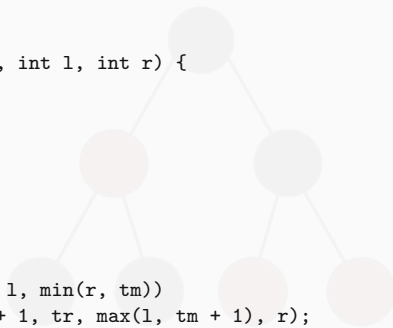
Range Assign

```
void assign(int v, int tl, int tr, int l, int r, int val) {  
    if (l > r)  
        return;  
    if (l == tl && r == tr) {  
        apply(v, tl, tr, val);  
        return;  
    }  
  
    push(v, tl, tr);  
    int tm = (tl + tr) / 2;  
  
    assign(v * 2, tl, tm, l, min(r, tm), val);  
    assign(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r, val);  
  
    t[v] = t[v * 2] + t[v * 2 + 1];  
}
```



Range Sum

```
int sum(int v, int tl, int tr, int l, int r) {  
    if (l > r)  
        return 0;  
    if (l == tl && r == tr)  
        return t[v];  
  
    push(v, tl, tr);  
    int tm = (tl + tr) / 2;  
  
    return sum(v * 2, tl, tm, l, min(r, tm))  
        + sum(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r);  
}
```



How One Query is Processed

Now the whole problem becomes systematic.

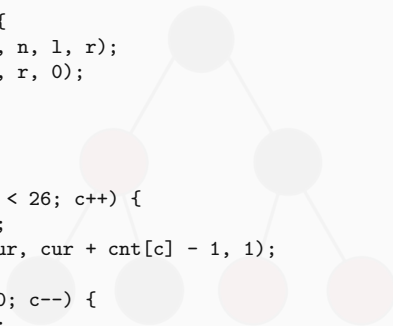
For a query (l, r, k) :

- 1 For each character c , compute $\text{cnt}[c] = \text{sum}_c(l, r)$
- 2 For each character tree, assign $[l, r]$ to 0
- 3 Rebuild the segment using blocks of characters:
 - if $k = 1$: from a to z
 - if $k = 0$: from z to a
- 4 If $\text{cnt}[c] = x$, assign the next block of length x to 1 in tree c

So the substring is rebuilt in sorted order using only **range sums + range assignments**.

High-Level Query Routine

```
for (int c = 0; c < 26; c++) {  
    cnt[c] = tree[c].sum(1, 1, n, 1, r);  
    tree[c].assign(1, 1, n, 1, r, 0);  
}  
  
int cur = 1;  
  
if (k == 1) for (int c = 0; c < 26; c++) {  
    if (cnt[c] == 0) continue;  
    tree[c].assign(1, 1, n, cur, cur + cnt[c] - 1, 1);  
    cur += cnt[c];  
} else for (int c = 25; c >= 0; c--) {  
    if (cnt[c] == 0) continue;  
    tree[c].assign(1, 1, n, cur, cur + cnt[c] - 1, 1);  
    cur += cnt[c];  
}
```



Why This Works

The correctness is driven by two simple facts:

- Querying all 26 trees gives the exact number of each character in the substring.
- After clearing $[l, r]$, we place back exactly the same total number of characters, only in a new order.

Therefore:

- the **multiset** of letters in the substring is preserved
- writing blocks from a to z gives non-decreasing order
- writing blocks from z to a gives non-increasing order

So every query performs exactly the required sort.

Complexity

For each query:

- 26 range-sum queries to count characters
- 26 range assignments to clear the interval
- up to 26 range assignments to rebuild it

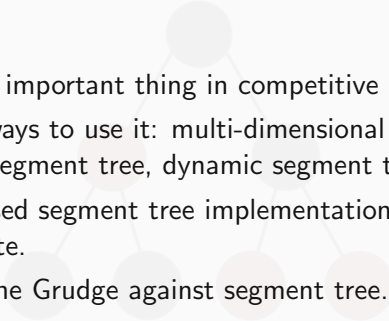
Each operation costs: $\mathcal{O}(\log n)$

So one query costs: $\mathcal{O}(26 \log n)$

Since the alphabet size is constant, this is effectively: $\mathcal{O}(\log n)$ per query

Overall: $\mathcal{O}(q \cdot 26 \log n)$, which is fast enough.

Takeaway: Segment Tree: Very Powerful TOOL

- 
- Segment tree is a very important thing in competitive programming.
 - There are very weird ways to use it: multi-dimensional segment trees, segment tree beats, persistent segment tree, dynamic segment tree, ...
 - You can have generalised segment tree implementations. Look at AtCoder Library for an abstract template.
 - Though, never have The Grudge against segment tree.